西南巡检项目说明书

此为我对西南巡检通过代码和沟通方式得到的汇总文档。

PIS对接

1. 代码位置,PIS同步使用webservice，客户端所需JAR包位置在WEB-INF/lib下。

> cn.jasgroup.jasframework.piswebserviceutil

2. 同步PIS需要的两把锁

- 系统集成号 SN
  系统集成号SN有系统集成号起始有效期和系统集成号终止有效期，如果系统集成号时间过期，则要重新获取系统集成号。

```
        /**
         *
         * 同步系统集成号
         * @param <oldSN> <String>
         *        <旧系统集成号>
         * @throws Exception
         */
        public static boolean SyncSN(String oldSN)throws Exception{
                String webServiceUrl =
ReadConfigUtil.getPlatformConfig("pis.systemSafetyService");
                Service serviceModel = new
ObjectServiceFactory().create(ISystemSafety.class);
                XFireProxyFactory serviceFactory = new
XFireProxyFactory(XFireFactory.newInstance().getXFire());
                ISystemSafety service = (ISystemSafety)
serviceFactory.create(serviceModel, webServiceUrl);
                String[] strs = (String[])service.GetSN(oldSN);
        if(strs.length>0){
                SN = strs[0];
                SNBD = strs[1];
                SNED = strs[2];
        }
        return true;
        }
```

- 数字令牌 DT
  数字令牌也有过期时间，如果数字令牌过期，则需要重新获取数字令牌 ，首先要判断系统集成号是否过期，如果过期，则重新获取系统集成号，否则直接获取数字令牌。

```
    /**
         * 同步数字令牌
```

```
         * @param sn
         * @throws Exception
         */
        public static boolean SyncDT(String sn) throws Exception{
        String webServiceUrl =
ReadConfigUtil.getPlatformConfig("pis.systemSafetyService");
        Service serviceModel = new
ObjectServiceFactory().create(ISystemSafety.class);
                XFireProxyFactory serviceFactory = new
XFireProxyFactory(XFireFactory.newInstance().getXFire());
                ISystemSafety service = (ISystemSafety)
serviceFactory.create(serviceModel, webServiceUrl);
                String[] strs = (String[])service.GetDT(sn);
        if(strs.length>0){
                DT = strs[0];
                DTBD = strs[1];
                DTED = strs[2];
        }
                return true;
        }
```

3. PIS同步数据包含以下几项。

- 同步管线数据
  `syncPineLoop()`
- 同步桩数据
  `syncMarker(String lineloopid)`
- 同步巡检范围
  `syncSubIns()`
- 同步不巡检范围
  `syncUnSubIns()`
- 同步部门
  `syncPisUnit()`
- 同步用户
  `syncPisUser()`
- 同步增量计划
  `syncIncInspectionPlan()`
- 同步全部计划
  `syncAllInspectionPlan()`

考核报表

综合评分计算说明

1. 分公司各站平均分数，得分四舍五入保留整数；
2. 红色警告：巡检完成率低于80%，需重点监察；
3. 蓝色警告：巡检完成率低于90%，需加强管理；

4. 管道线路巡检计划覆盖率=（制定巡检计划的管道里程/部门及子孙部门下该管线的管理范围里程-不巡检范围里程）×100%；低于100%以红色展示；
5. 管道线路巡检任务覆盖率=（落实巡检任务的管道里程/部门及子孙部门下该管线的管理范围里程-不巡检范围里程）×100%；低于100%以红色展示；
6. 管道线路巡检完成率=各分公司所辖站队总成绩/各分公司所辖巡检站队总数；
7. 巡检范围管理 == 部门管理范围

部门考核报表，此属于巡线工考核报表或者管道工考核的一级展示页面

> 部门考核报表也是一个定时任务，代码位置：
> cn.jasgroup.jasframework.linepatrolmanage.assessments.service.StatisticsScoreService.java
> 主要代码部分 statisticsScoreUnit(String statisticsdate) 方法，参数为需要统计的日期。

```
        /**
        *
         *部门分数统计
         *@param statisticsdate 统计日期
        */
       private void statisticsScoreUnit(String statisticsdate){
               String date = statisticsdate;
               //获取顶层部门
               Unit unit = statisticsScoreDao.queryTopUnit();
               //获取子部门
               List<Unit> childrenList =
statisticsScoreDao.queryUnit(unit.getOid());
               //删除三天前数据
               statisticsScoreDao.deleteStationScore(statisticsdate);
               for(int j=0;j<3;j++){
                       date = DateTime.getDateChange(statisticsdate, -j);
                       System.out.println(date);
                       System.out.println(j);
                       for(int i=0;i<childrenList.size();i++){
                               //统计结果列表
                               List<GpsUnitScore> scoreList = new
ArrayList<GpsUnitScore>();
                               Unit u = childrenList.get(i);
                               //计算场站分数（巡线工）
                               this.queryUnitChildren(u.getOid(),date,scoreList);
                               //计算分公司分数（巡线工）
                               this.statisticsScoreFiliale(u,scoreList, date);
                               //保存结果
                               gpsUnitScoreService.batchSave(GpsUnitScore.class,
scoreList);
                       }
               }
       }
```

概要

- 首先获取顶层部门以及其下的所有子部门，接着删除数据，此处需解答,调用的部分写的是删除三天前，实现部分的注释写的是三天内，但是sql中的含义应该是三天之后的所有天数。
    - 以参数统计时间为最大时间，依次向前推一天，并且循环所有子部门，利用递归获取到所有部门的分数，关键查询的表是 GSP_STATION_FSTATISTIC 和 GPS_INSPECTOR_SCORE。
- 巡检计划覆盖率和巡检任务覆盖率计算 ，最后保存在表GSP_STATION_FSTATISTIC中
    - 通过部门递归获取到所有部门
    - 通过部门获取到该部门的管理范围，部门管理范围有多条
    - 循环所有的部门管理范围，通过部门id和范围中的管线id获取到部门以及子孙部门下每条管线的不巡检范围,取相同管线下不巡检范围的并集。
    - 通过部门id和管线id 获取巡检计划范围,取相同管线下巡检计划范围的并集,去除在巡检计划范围内存在的不巡检范围
    - 通过部门id和管线id巡检任务范围,取相同管线下巡检任务范围的并集,去除在巡检任务范围内存在的不巡检范围
    - 代码位置:

> cn.jasgroup.jasframework.statisticsform.stationfstatistic.service.StationfstatisticService.java

    - 覆盖率是根据，每个部门的每条管线的 有生产任务的计划的区段长度/（管理范围长度(GPS_SUBINS)-不巡检范围长度），这是计划覆盖率，任务就换成任务中的区段长度
      比如，在方法saveStationfstatistic的末尾有一段代码计算了此覆盖率：

```
/* 计算巡检计划覆盖率 */
plancoveragerate =
planlinelength.divide(subinslength.subtract(unsubinslength), 6,
BigDecimal.ROUND_HALF_UP);
/* 计算巡检任务覆盖率 */
taskcoveragerate =
instasklength.divide(subinslength.subtract(unsubinslength), 6,
BigDecimal.ROUND_HALF_UP);
```

    - 管理范围的长度 = 管理范围终止里程 - 管理范围起始里程
    - 分别计算出如下3项：
        - 部门及子孙部门下某一条管线下的不巡检范围
        - 部门及子孙部门下某一条管线下在管理范围内的巡检计划范围
        - 部门及子孙部门下某一条管线下在管理范围内的巡检任务范围
    - 覆盖率是针对于巡线工而言的，管道工是没有覆盖率的。
    - 覆盖率统计每天定时执行，计算当天任务覆盖率，在任务生成以后进行。
    - 如果是分公司不查询分公司自己
    - 定时执行的巡检覆盖率的大纲方法如下：

```
/**
 * 保存巡检覆盖率统计数据（仅巡线工）
 */
public void saveStationfstatistic(){
        //获取当前日期
```

```
        String statisticsdate = "2018-09-29";//DateTime.getDate();
        //获取顶层部门
        Unit unit = statisticsScoreDao.queryTopUnit();
        //获取分公司
        List<Unit> childrenList =
statisticsScoreDao.queryUnit(unit.getOid());
        for(int i=0;i<childrenList.size();i++){
                Unit u = childrenList.get(i);
                //计算场站巡检覆盖率（巡线工）
                List<Stationfstatistic> statisticList =
this.queryUnitChildren(u.getOid(), statisticsdate);
                //计算分公司巡检覆盖率（巡线工）
                this.saveSubDeptfstatistic(u, statisticList, statisticsdate,
unit.getOid());
        }

}
```

- 巡检范围 3类，不巡检范围，计划巡检范围以及巡检任务范围，sql如下：

```
/**
 * 部门及子孙部门下某一条管线下在管理范围内的不巡检范围（仅大、小站）
 * @param hierarchy
 * @param lineloopoid
 * @param statisticsdate
 * @return
 */
public List<GpsUnsubinspection> queryUnSubinspection(String hierarchy,
String lineloopoid, String statisticsdate){
        String sql = "select distinct
beginstation,endstation,beginlocation,endlocation from (\n";
        String subdept_hierarchy = hierarchy.substring(0,
hierarchy.lastIndexOf("."));//分公司层级
        String parent_hierarchy = subdept_hierarchy;//上一级部门层级
        if(StringUtils.countMatches(hierarchy,".") == 4){//小站
                subdept_hierarchy = parent_hierarchy.substring(0,
parent_hierarchy.lastIndexOf("."));
        }

        sql += "select distinct case when t2.beginstation > t1.beginstation
then t2.beginstation else t1.beginstation end beginstation,\n" +
                "case when t2.endstation < t1.endstation then t2.endstation
else t1.endstation end endstation,\n" +
                "case when t2.beginstation > t1.beginstation then
t2.beginlocation else t1.beginlocation end beginlocation,\n" +
                "case when t2.endstation < t1.endstation then t2.endlocation
else t1.endlocation end endlocation\n" +
                "from\n" +
                "(select t.* from GPS_UNSUBINSPECTION t\n" +
                "inner join pri_unit t1 on t.unitid=t1.oid and t1.active=1
and t1.ispatrol=1 and (t1.hierarchy = '"+subdept_hierarchy+"' or
t1.hierarchy = '"+parent_hierarchy+"')\n" +
```

```
                "and t.lineloopoid='"+lineloopoid+"' and t.active=1\n" +
                ") t1 inner join\n" +
                "(select t1.* from GPS_SUBINS t1 inner join pri_unit t2 on
t1.unitid=t2.oid and t2.active=1 where t1.active=1 and t2.hierarchy =
'"+hierarchy+"') t2\n" +
                "on t2.lineloopoid=t1.lineloopoid\n" +
                "where '"+statisticsdate+"' between
to_char(t1.begindate,'yyyy-MM-dd') and (case when t1.enddate is null then
'9999-12-01' else to_char(t1.enddate,'yyyy-MM-dd') end)\n" +
                "and (t1.repealdate is null or '"+statisticsdate+"' <
to_char(t1.repealdate,'yyyy-MM-dd'))\n" +
                "and (t1.beginstation between t2.beginstation and
t2.endstation or t1.endstation between t2.beginstation and t2.endstation)\n"
+
                "union all\n" +
                "select distinct case when t2.beginstation > t.beginstation
then t2.beginstation else t.beginstation end beginstation,\n" +
                "case when t2.endstation < t.endstation then t2.endstation
else t.endstation end endstation,\n" +
                "case when t2.beginstation > t.beginstation then
t2.beginlocation else t.beginlocation end beginlocation,\n" +
                "case when t2.endstation < t.endstation then t2.endlocation
else t.endlocation end endlocation\n" +
                "from GPS_UNSUBINSPECTION t\n" +
                "inner join pri_unit t1 on t.unitid=t1.oid and t1.active=1
and t1.ispatrol=1\n" +
                "inner join GPS_SUBINS t2 on t.unitid=t2.unitid and
t2.lineloopoid=t.lineloopoid and t2.active=1\n" +
                "where t.active=1 and t1.hierarchy like '"+hierarchy+"%' and
t.lineloopoid='"+lineloopoid+"'\n" +
                "and '"+statisticsdate+"' between to_char(t.begindate,'yyyy-
MM-dd') and (case when t.enddate is null then '9999-12-01' else
to_char(t.enddate,'yyyy-MM-dd') end)\n" +
                "and (t.repealdate is null or '"+statisticsdate+"' <
to_char(t.repealdate,'yyyy-MM-dd'))\n" +
                "and (t.beginstation between t2.beginstation and
t2.endstation or t.endstation between t2.beginstation and t2.endstation)\n";
        sql += ")order by beginstation";
        List<GpsUnsubinspection> list =
this.baseJdbcTemplate.queryForList(sql, null, GpsUnsubinspection.class);
        return list;
}

/**
 * 部门及子孙部门下某一条管线下在管理范围内的巡检计划范围
 * @param hierarchy
 * @param lineloopoid
 * @param statisticsdate
 * @return
 */
public List<GpsPlanLineInfo> queryPlanLineInfo(String hierarchy, String
lineloopoid, String statisticsdate){
        String sql = "select distinct case when t3.beginstation >
t2.beginstation then t3.beginstation else t2.beginstation end
```

```
beginstation,\n" +
                                "case when t3.endstation < t2.endstation
then t3.endstation else t2.endstation end endstation,\n" +
                                "case when t3.beginstation > t2.beginstation
then t3.beginlocation else t2.beginlocation end beginlocation,\n" +
                                "case when t3.endstation < t2.endstation
then t3.endlocation else t2.endlocation end endlocation\n" +
                                "from gps_plan_info t\n" +
                                "inner join gps_plan_line t2 on
t.oid=t2.planeventid and t2.active=1\n" +
                                "inner join gps_instask_day t4 on
t.oid=t4.planevoid and t4.active=1\n" +
                                "inner join pri_unit t1 on
t4.execunitid=t1.oid and t1.active=1 and t1.ispatrol=1\n" +
                                "inner join GPS_SUBINS t3 on
t4.execunitid=t3.unitid and t3.lineloopoid=t2.lineloopoid and t3.active=1\n"
+
                                "where t.active=1 and t.planflag='01' and
t.inspectortype='01' and t1.hierarchy like '"+hierarchy+"%'\n" +
                                "and t2.lineloopoid='"+lineloopoid+"'\n" +
                                "and '"+statisticsdate+"' between
to_char(t.insbdate,'yyyy-MM-dd') and (case when t.insedate is null then
'9999-12-01' else to_char(t.insedate,'yyyy-MM-dd') end)\n" +
                                "and (t.repealdate is null or
'"+statisticsdate+"' < to_char(t.repealdate,'yyyy-MM-dd'))\n" +
                                "and (t2.beginstation between
t3.beginstation and t3.endstation or t2.endstation between t3.beginstation
and t3.endstation\n" +
                                "or t3.beginstation between t2.beginstation
and t2.endstation or t3.endstation between t2.beginstation and
t2.endstation)\n" +
                                "order by case when t3.beginstation >
t2.beginstation then t3.beginstation else t2.beginstation end";
        List<GpsPlanLineInfo> list = this.baseJdbcTemplate.queryForList(sql,
null, GpsPlanLineInfo.class);
        return list;
}

/**
 * 部门及子孙部门下某一条管线下在管理范围内的巡检任务范围
 * @param hierarchy
 * @param lineloopoid
 * @param statisticsdate
 * @return
 */
public List<GpsInstaskDayLine> queryInstask(String hierarchy, String
lineloopoid, String statisticsdate){
        String sql = "select distinct case when t3.beginstation >
t2.beginstation then t3.beginstation else t2.beginstation end
beginstation,\n" +
                                "case when t3.endstation < t2.endstation
then t3.endstation else t2.endstation end endstation,\n" +
                                "case when t3.beginstation > t2.beginstation
then t3.beginlocation else t2.beginlocation end beginlocation,\n" +
```

```
                                "case when t3.endstation < t2.endstation
then t3.endlocation else t2.endlocation end endlocation\n" +
                                "from gps_instask_day t\n" +
                                "inner join pri_unit t1 on
t.execunitid=t1.oid and t1.active=1 and t1.ispatrol=1\n" +
                                "inner join gps_instask_day_line t2 on
t.oid=t2.instaskoid and t2.active=1\n" +
                                "inner join GPS_SUBINS t3 on
t.execunitid=t3.unitid and t3.lineloopoid=t2.lineloopoid and t3.active=1\n"
+
                                "where t.active=1 and t.inspectortype='01'
and t1.hierarchy like '"+hierarchy+"%' and
t2.lineloopoid='"+lineloopoid+"'\n" +
                                "and '"+statisticsdate+"' between
to_char(t.insbdate,'yyyy-MM-dd') and to_char(t.insedate,'yyyy-MM-dd')\n" +
                                "and (t2.beginstation between
t3.beginstation and t3.endstation or t2.endstation between t3.beginstation
and t3.endstation)\n" +
                                "order by case when t3.beginstation >
t2.beginstation then t3.beginstation else t2.beginstation end";
        List<GpsInstaskDayLine> list =
this.baseJdbcTemplate.queryForList(sql, null, GpsInstaskDayLine.class);
        return list;
}
```

- 获取场站分数是查询表 GPS_INSPECTOR_SCORE，具体涉及的分数有，巡检完成率,考核分数,应巡关键点数,实巡关键点数,应巡临时关键点数,实巡临时关键点数,轨迹线匹配度.
    - 分公司的分数是场站分数汇总后的平均值。
    - GPS_INSPECTOR_SCORE 是如何生成的呢，具体参考巡线工考核报表。

巡线工考核报表

测试更新

管道工考核报表

> 管道工统计是一个定时任务，代码位置：cn.jasgroup.jasframework.linepatrolmanage.assessments.service.StatisticsScoreService.java 中的 statisticsScoreG 方法。只需执行此方法就可以吧统计完的数据存在表gps_inspector_score中。

```
        /*****************************************************管道工统计 开始
*******************************************************/
        /**
         *
         * 管道工分数统计
         */
        public void statisticsScoreG(String nowDate){
                inspectortype = "02";
                //开始时间
                String beginDate = DateTime.getDateChange(nowDate, -7);
```

```
                //结束时间
                String endDate = DateTime.getDateChange(nowDate, -1);

                //获取全部任务部门
                List<Unit> unitList = statisticsScoreDao.queryUnit(null);

                //分部门计算巡检人员分数
                try {
                        for (int i = 0; i < unitList.size(); i++){
                                Unit u = unitList.get(i);
                                System.out.println(u.getUnitName());
                                int bufferwidth =
gpsBufferWidthService.getBufferwidth(u.getOid(), 0);
                                this.queryScore(endDate, u.getOid(),bufferwidth);
                        }
                } catch (Exception e) {
                        e.printStackTrace();
                }
                //部门分数计算（管道工）
                statisticsScoreUnit(endDate);
        }
        /*****************************************************管道工统计 结束
*****************************************************/
}
```

概要

- 首先获取到所有的部门，（根据每个部门获取到该部门下的所有人员的巡检分数。）
  - 根据部门获取到该部门的缓冲带宽度。（缓冲带宽度涉及的表gps_buffer_width）
  - 按照统计时间，部门ID和部门缓冲范围获取该部门下的所有管道工人员
  - 计算每位管道工人员的分数
    - 通过调用queryLineLocation方法获取到该人员的所有得分情况
    - 将每位管道工人员的得分情况保存到表gps_inspector_score中

具体实现步奏

- 获取管道工人员的考核分数关键性代码如下。

1. 关键步奏就是获取到此人的巡检任务，巡检任务中包含了巡检时间和巡检关键点信息，因为巡检时间和巡检关键点都是分开表存储，所以取值的时候有点麻烦。

- 获取每个人员的巡检任务 （巡检任务表 gps_instask_day ）
  - 获取巡检任务时间 （ gps_instask_day_line_time ）
    - 通过巡检任务时间找到巡检任务关键点（ gps_instask_day_point ）

2. 获取到每位管道工的临时任务（ `GPS_TEMPORARY_TASK` ）
3. 循环管道工所有的任务数据
4. 关键点分数计算， 实际巡检的关键点数量 / 应该巡检的关键点数量
   临时关键点的计算，同上

5. 计算巡检完成率（轨迹线匹配度占比0%，关键点占比90%，临时关键点占比10%）
   巡检完成率 = 轨迹线匹配度 * 0 + 关键点分数 * 0.9 + 临时关键点分数 * 0.1 ，巡检完成率为0视为异常数据
6. 计算轨迹线的匹配度

- 通过任务管线的集合获取到标准的轨迹线点
  - 通过管线ID和巡检人员ID获取到巡检管线上的所有关键点。（ 涉及的表有GPS_PATHLINE_MAIN, GPS_PATHLINE_XYPOINT ）
  - 拼接标准轨迹线点转换成线对象 Polyline
  - 通过调用我们系统封装的lengthRadioOfIntersectedPaths 方法 传入标准轨迹线和巡检gps轨迹线的线对象以及缓冲区的宽度，得到轨迹线与标准轨迹线缓冲区轨迹重叠的长度占比.
    - 首先要根据人员和管线获取到该人员要巡检的标准管线点。
    - 根据任务的时间分别获取到不同的时间段内人员巡检的实际轨迹线。
    - 通过任务时间段内，此人员的实际轨迹线和标准轨迹线通过arcgis服务提供的方法计算出重叠的长度占比。(多天一巡和一天多巡的方式有点区别。但是最后得到的占比是相对于每一条管线的。)

  注意：此处代码需要验证，如果是一天多巡，就是每个时间段计算一次标准轨迹点与实际巡检轨迹点的匹配度，然后最终的匹配度是除以巡检时间的数量取平均值；如果是多天一巡，就把该时间段内的所有巡检点组成一条线对象，最终计算与标准轨迹线的匹配度，多天一巡匹配度的计算需要奎斌解答。

```
/**
         * 获取巡检人员 考核数据
         *  @param statisticsdate 统计日期
         *  @param insoid 巡检人员
         *  @param deviceoid 设备标识
         *  @param unitid 巡检人员部门
         *  @param scoreList 统计结果集合
         */
        private void queryLineLocation(String statisticsdate,String insoid,String
deviceoid,String unitid,List<GpsInsScore> scoreList,int bufferwidth) throws
Exception{

                //获取巡检人员任务数据
                List<GpsSyncTaskBo> taskList = syncTaskService.syncTask(insoid,
statisticsdate);
                //轨迹线匹配度
                double pathlinerate = 0;
                //应巡关键点数
                int ykeypointnum = 0;
                //实巡关键点数
                int skeypointnum = 0;

                //获取巡检人员临时关键点任务
                List<Map<String,Object>> temporarytaskList =
statisticsScoreDao.queryTemporarytask(insoid, statisticsdate);
```

```
                //应巡临时关键点数
                int ytemporarykeypointnum = 0;
                //实巡临时关键点数
                int stemporarykeypointnum = 0;

                //如果有任务数据继续执行以下代码
                if((taskList != null && taskList.size() > 0) || (temporarytaskList
!= null && temporarytaskList.size() > 0)){
                        //循环任务计算巡检完成率和考核分数
                        for(int i=0;i<taskList.size();i++){
                                GpsSyncTaskBo bo = taskList.get(i);
                                //获取巡检任务时间段集合
                                List<GpsSyncTaskTimeBo> timeList =
bo.getTasktime();
                                //获取巡检任务相关线路
                                List<GpsInstaskDayLine> lineList =
statisticsScoreDao.getTaskLine(bo.getOid());
                                //计算轨迹线匹配度
                                pathlinerate =
this.getTaskCompletionrate(timeList,
lineList,bo.getInspectorid(),bo.getInsfrequnitval(),bufferwidth);

                                //insfrequnitval 大于1为多天一巡
                                if(bo.getInsfrequnitval() > 1){
                                        //获取任务关键点已打点和未打点数据
                                        Map<String,Object> taskKeyPointMap =
statisticsScoreDao.queryTaskKeyPointStatusDays(bo.getOid());
                                        if(taskKeyPointMap != null){
                                                skeypointnum =
Integer.parseInt(taskKeyPointMap.get("s")+"");
                                                ykeypointnum = skeypointnum +
Integer.parseInt(taskKeyPointMap.get("w")+"");
                                        }
                                }else{
                                        //获取任务关键点已打点和未打点数据
                                        Map<String,Object> taskKeyPointMap =
statisticsScoreDao.queryTaskKeyPointStatus(bo.getOid());
                                        if(taskKeyPointMap != null){
                                                skeypointnum =
Integer.parseInt(taskKeyPointMap.get("s")+"");
                                                ykeypointnum = skeypointnum +
Integer.parseInt(taskKeyPointMap.get("w")+"");
                                        }
                                }
                        }

                        //循环临时关键点任务
                        for(int i=0;i<temporarytaskList.size();i++){
                                Map<String,Object> temporarytask =
temporarytaskList.get(i);
                                //巡检频次
                                int insfreq = temporarytask.get("insfreq") == null
? 0 : Integer.parseInt(temporarytask.get("insfreq")+"");
                                //临时关键点任务oid
```

```
                                String temporarytaskoid =
temporarytask.get("oid")+"";
                                //insfreq 第一个数字表示几天，第二个数字表示几巡
                                if(insfreq >= 21){
                                        //获取任务关键点已打点和未打点数据
                                        Map<String,Object> taskKeyPointMap =
statisticsScoreDao.queryTemporaryKeyPointStatusDays(temporarytaskoid);
                                        if(taskKeyPointMap != null){
                                                stemporarykeypointnum =
Integer.parseInt(taskKeyPointMap.get("s")+"");
                                                ytemporarykeypointnum =
skeypointnum + Integer.parseInt(taskKeyPointMap.get("w")+"");
                                        }
                                }else{
                                        //获取任务关键点已打点和未打点数据
                                        Map<String,Object> taskKeyPointMap =
statisticsScoreDao.queryTemporaryKeyPointStatus(temporarytaskoid);
                                        if(taskKeyPointMap != null){
                                                stemporarykeypointnum =
Integer.parseInt(taskKeyPointMap.get("s")+"");
                                                ytemporarykeypointnum =
stemporarykeypointnum + Integer.parseInt(taskKeyPointMap.get("w")+"");
                                        }
                                }
                        }

                        //获取人员巡检区段
                        List<GpsInsrange> list =
statisticsScoreDao.queryLineLocation(insoid);
                        //起始位置
                        String blocation = "";
                        //终止位置
                        String elocation = "";
                        //巡线里程
                        double linelength = 0;

                        for(int i=0;i<list.size();i++){
                                GpsInsrange ge = list.get(i);
                                if(i == list.size()-1){
                                        blocation += ge.getBeginlocation();
                                        elocation += ge.getEndlocation();
                                }else{
                                        blocation += ge.getBeginlocation() + ",";
                                        elocation += ge.getEndlocation() + ",";
                                }
                                //计算巡线里程
                                linelength += ge.getEndstation() -
ge.getBeginstation();
                        }

                        //巡检完成率
                        double completionrate = 0 ;

                        //关键点分数计算(实巡关键点/应巡关键点)
```

```
                        double keyPointScore = skeypointnum / ykeypointnum;
                        keyPointScore =
Double.parseDouble(String.format("%.2f",keyPointScore * 100));

                        //临时关键点分数计算(实巡临时关键点/应巡临时关键点)
                        double temporaryKeyPointScore = stemporarykeypointnum /
ytemporarykeypointnum;
                        temporaryKeyPointScore =
Double.parseDouble(String.format("%.2f",temporaryKeyPointScore * 100));

                        //计算巡检完成率（轨迹线匹配度占比0%，关键点占比90%，临时关键点
占比10%）
                        completionrate = (pathlinerate * 0) + (keyPointScore *
0.9) + (temporaryKeyPointScore * 0.1);

                        //巡检完成率为0视为异常数据
                        if(completionrate == 0){
                                //保存异常数据
                        }else{
                                //保存正常记录
                        }

                        GpsInsScore insscore = new GpsInsScore();
                        //统计时间

insscore.setStatisticsdate(DateTime.getDateFromDateString(statisticsdate, "yyyy-
MM-dd"));
                        //巡检人员
                        insscore.setInspectorid(insoid);
                        //设备标识
                        insscore.setDeviceoid(deviceoid);
                        //起始位置
                        insscore.setBeginlocation(blocation);
                        //结束位置
                        insscore.setEndlocation(elocation);
                        //巡线里程
                        insscore.setLinelength(linelength);
                        //巡检完成率
                        insscore.setCompletionrate(completionrate);
                        //巡检人员类型
                        insscore.setInspectortype(inspectortype);
                        //巡检人员部门
                        insscore.setUnitid(unitid);
                        //轨迹线匹配度
                        insscore.setPathlinerate(pathlinerate);
                        //应巡关键点数
                        insscore.setYkeypointnum(ykeypointnum);
                        //实巡关键点数
                        insscore.setSkeypointnum(skeypointnum);
                        //应巡临时关键点数
                        insscore.setYtemporarykeypointnum(ytemporarykeypointnum);
                        //实巡临时关键点数
                        insscore.setStemporarykeypointnum(stemporarykeypointnum);
                        scoreList.add(insscore);
```

```
                }
        }
```

定时器

1. 同步异常处理结果定时器
   SyncErrorRecordService
2. 删除管道工巡检区段定时器
   cn.jasgroup.jasframework.linepatrolmanage.subsystem.insrangemanage.service.GpsInsrangeService void deleteGInsrange()
3. 生成巡检任务定时器
   - 根据巡检计划生成管道工巡检任务和巡线工巡检任务
     - cn.jasgroup.jasframework.linepatrolmanage.instask.instaskday.service.GpsInstaskDayService void doInstask();
   - 生成临时任务
     - cn.jasgroup.jasframework.linepatrolmanage.instask.temporarytask.service.GpsTemporaryTaskService void generateTemporaryTask();
4. 同步系统集成号定时器
   - cn.jasgroup.jasframework.piswebserviceutil下的所有方法
5. 统计报表定时器
   - 巡线工分数统计
     - cn.jasgroup.jasframework.linepatrolmanage.assessments.StatisticsScoreService void statisticsScore();
   - 管道工分数统计
     - cn.jasgroup.jasframework.linepatrolmanage.assessments.StatisticsScoreService void statisticsScoreG();
   - 部门分数统计
     - cn.jasgroup.jasframework.linepatrolmanage.assessments.StatisticsScoreService void statisticsScoreUnit();

巡检计划

巡检计划定时生成巡检任务

1. 代码位置：

> cn.jasgroup.jasframework.linepatrolmanage.instask.instaskday.service.GpsInstaskDayService void doInstask(String inspectortype);
> 参数
> inspectortype : 01:巡线工，02：管道工。